FINAL YEAR PROJECT EVALUATION & VIVA DOCUMENTATION

Final Viva Examination & Defense Preparation

Comprehensive 50 Questions & Answers, Methodological Deep Dives, and Examiner Scenarios

Project: Academic Project Novelty Detection **Architecture:** Weighted Section TF-IDF + Dense Semantic (all-MiniLM-L6-v2)

Tuned Alpha: $\alpha = 0.60$ **Status:** Verified & Ready for Viva

Academic Project Novelty Detection System

Final Viva Examination Preparation & Q&A Guide

Target Audience: B.Tech Computer Science & Engineering Students
Subject: Natural Language Processing, Machine Learning, Web Engineering, and Explainable AI

Table of Contents

1. Core Conceptual Foundations (Q1–Q6)
2. NLP & Feature Extraction (Q7–Q14)
3. Model Architecture, Fusion & Calibration (Q15–Q22)
4. Explainable AI & Scoring Mechanics (Q23–Q27)
5. Software Architecture & Engineering (Q28–Q34)
6. Evaluation Metrics & Scientific Protocol (Q35–Q40)
7. Challenging External Examiner Questions (Q41–Q50)

---

1. Core Conceptual Foundations

Q1: What is the fundamental difference between Novelty Detection, Similarity Detection, and Plagiarism Detection?

- **Plagiarism Detection**: Focuses on finding **unauthorized verbatim copying or near-verbatim borrowing of text strings** without proper citation. It scans across general web pages or papers regardless of academic structure.
- **Similarity Detection**: Measures the **lexical or semantic overlap** between two texts. Two proposals may have high similarity because they both work in healthcare with deep learning, even if their research goals are completely distinct.
- **Novelty Detection**: Evaluates whether a proposal introduces **original, non-redundant scientific contributions** compared to a known baseline repository. A proposal can use identical tools or domains, but if it applies them to a novel dataset or solves a distinct problem, it possesses high novelty. Novelty is inversely related to maximum historical similarity: ***INLINE_MATH_5***.

Q2: What is the primary problem your project aims to solve?

In universities, hundreds of student capstone and mini-project proposals are submitted each semester. Faculty review panels struggle to remember past projects across several academic years. Students frequently submit projects

that are near-identical to previous batches under reworded titles. Our project automatically evaluates a proposal across 8 distinct sections, compares it against an indexed historical corpus, assigns a calibrated novelty score, and explains exactly why and where any duplication exists.

Q3: Why can't we just use Turnitin or Google Search for this task?

1. Commercial plagiarism checkers treat the entire document as a single flat string of text. They do not parse structured academic sections like Dataset, Methodology, or Objectives.
2. They rely primarily on verbatim string and n-gram matching, missing conceptual paraphrasing.
3. They search the public web, whereas institutional project duplication occurs against internal departmental archives.
4. They provide percentage match scores without explaining the specific technical drivers of academic redundancy.

Q4: Why is your system section-aware instead of evaluating the proposal as a single document?

Different sections of an academic proposal have vastly different impacts on novelty:

- An identical **Dataset** (***INLINE_MATH_6*** weight) implies that students are re-running existing benchmarks without new data.
- The **Abstract** or **Objectives** often contain generic academic filler ("*This project proposes a framework to improve accuracy...*"). Evaluating sections independently prevents generic boilerplate from inflating the similarity score, and allows faculty to see whether overlap is merely in tools or in the actual research problem.

Q5: What is the reference corpus used in this project?

The reference corpus comprises **600 structured academic projects** spanning 7 core computer science domains (AI/ML, Data Science, Cyber Security, Healthcare Informatics, Cloud, IoT, and Web). Each record contains all 8 required textual fields plus metadata (`project_id` , `domain` , `academic_year`).

Q6: Does a high novelty score in your system prove that a project is completely original?

No. Novelty is strictly **corpus-relative** to the 600 projects indexed in the local reference database. It means the proposal does not duplicate prior projects in that repository. It does not establish worldwide academic originality across all published literature.

---

2. NLP & Feature Extraction

Q7: What is TF-IDF and what does it measure?

TF-IDF stands for **Term Frequency-Inverse Document Frequency**:

- **Term Frequency (TF)**: How frequently a term appears in a given section.
- **Inverse Document Frequency (IDF)**: Measures how common or rare a term is across all 600 projects in the corpus: ***INLINE_MATH_7***. It assigns high numerical importance to technical, section-specific terms (e.g., *PlantVillage*, *Ryu Controller*) while giving low weights to common words.

Q8: What is sublinear term frequency scaling and why did you use it?

Standard TF counts terms linearly (***INLINE_MATH_8***). In academic writing, a student who writes the word "*CNN*" 10 times in a methodology is not 10 times more similar to another project than someone who writes it once. Sublinear TF applies the logarithmic transformation:

BLOCK_MATH_0

This dampens the effect of repetitive keywords and prevents artificial similarity inflation.

Q9: Why did you include bigrams in addition to unigrams in TF-IDF?

Unigrams treat words individually. However, in computer science, meaning often resides in compound technical phrases:

- "*random forest*" vs. "*random*" and "*forest*"
- "*convolutional network*" vs. "*network*"
- "*credit card*" vs. "*card*" Using an n-gram range of ***INLINE_MATH_9*** captures these meaningful pairs, significantly improving match precision.

Q10: How is Cosine Similarity mathematically calculated between two vectors?

Cosine similarity measures the cosine of the angle between two multi-dimensional vectors in feature space:

BLOCK_MATH_1

It ranges from ***INLINE_MATH_10*** (orthogonal/completely different) to ***INLINE_MATH_11*** (identical direction/highest similarity), and is length-invariant.

Q11: What is a dense semantic embedding and how does it differ from a sparse TF-IDF vector?

- **Sparse TF-IDF Vector**: Dimension equals the vocabulary size (e.g., thousands of words). Most entries are zeros. It only records exact word matches; "*pathology*" and "*disease*" have zero overlap.
- **Dense Embedding**: A low-dimensional continuous vector (e.g., 384 dimensions) generated by a pre-trained transformer. Every dimension contains a continuous float representing semantic meaning. In dense space, "*crop disease*" and "*botanical affliction*" have a high cosine similarity because they appear in similar contextual relationships.

Q12: Which transformer model did you select for dense embeddings and why?

We selected `sentence-transformers/all-MiniLM-L6-v2`:

- **Dimensions**: 384-dimensional dense vectors.
- **Layers**: 6 transformer layers, 22.7M parameters.
- **Rationale**: It provides an ideal balance of state-of-the-art semantic sentence clustering while being compact enough to run sub-second CPU inference on commodity hardware without requiring expensive GPU servers.

Q13: Why did you clamp negative cosine values in the semantic branch?

Raw cosine similarity between dense embeddings can occasionally produce negative values (down to ***INLINE_MATH_12***) for opposing or unrelated text. In an academic similarity metric, similarity cannot be negative—two completely different projects simply share zero similarity (***INLINE_MATH_13***). We clamp negative semantic cosines using ***INLINE_MATH_14*** to enforce monotonic bounds in ***INLINE_MATH_15***.

Q14: What text preprocessing steps do you perform before vectorization?

1. Conversion to lowercase string.
2. Stripping control characters and non-printable bytes.
3. Preserving hyphens in technical terms (*ResNet-50*, *TF-IDF*).
4. English stop-word filtering during TF-IDF vectorization.
5. Minimum length validation across each of the 8 required sections.

---

3. Model Architecture, Fusion & Calibration

Q15: What is the Convex Hybrid Fusion formula in your system?

For each section ***INLINE_MATH_16***, the hybrid similarity combines lexical and semantic cosines:

BLOCK_MATH_2

Where ***INLINE_MATH_17*** is the weighting factor.

Q16: How was the optimal alpha value (***INLINE_MATH_18***) determined?

We conducted a grid search across ***INLINE_MATH_19*** using **5-Fold Cross-Validation on the 30 `DEV_CALIBRATION` faculty-annotated pairs**:

- ***INLINE_MATH_20*** achieved the lowest out-of-fold RMSE (***INLINE_MATH_21***).
- Mean Bias Error was near zero (***INLINE_MATH_22***), avoiding the positive overprediction of lower alphas and the negative underprediction of higher alphas.
- Cross-validation exact agreement reached ***INLINE_MATH_23***.

Q17: What was wrong with the baseline Ridge regression model from Phase 2?

The unconstrained Ridge model fitted unconstrained coefficients and an intercept (***INLINE_MATH_24***). This created three fatal defects:

1. **Negative Weights**: Sections like Problem Statement (***INLINE_MATH_25***) and Technology (***INLINE_MATH_26***) had negative coefficients. A student who shared *more* technology would receive a *lower* similarity score!
2. **Under-reaching Ceiling**: For an exact copy where all similarities were ***INLINE_MATH_27***, the predicted score was only ***INLINE_MATH_28*** instead of ***INLINE_MATH_29*** because the weights did not sum to ***INLINE_MATH_30***.
3. **Negative Intercept**: Highly novel projects with zero overlap produced negative raw similarity predictions (***INLINE_MATH_31***).

Q18: How does your Candidate Constrained model solve these issues?

We used Sequential Least Squares Programming (SLSQP) to enforce:

1. **Non-negativity**: ***INLINE_MATH_32***.
2. **Exact Sum Constraint**: ***INLINE_MATH_33***.
3. **Zero Intercept**: ***INLINE_MATH_34***. This guarantees that an exact copy reaches exactly ***INLINE_MATH_35*** similarity (***INLINE_MATH_36*** novelty), completely novel projects receive ***INLINE_MATH_37*** similarity (***INLINE_MATH_38*** novelty), and every section's contribution is strictly non-negative.

Q19: Why are four of the section weights equal to 0.0?

In our candidate model, `Abstract`, `Problem Statement`, `Objectives`, and `Algorithm/Model` received zero weights:

- Students across different projects often use identical phrasing in abstracts and objectives ("*The primary objective is to develop a machine learning model...*").
- Popular algorithms like *CNN*, *YOLO*, or *Random Forest* appear in thousands of unrelated projects.
- Assigning weights to these sections produced false-positive similarity spikes. The model focuses its weights where genuine academic novelty is defined: **Dataset (***INLINE_MATH_39***)**, **Title (***INLINE_MATH_40***)**, **Technology & Tools (***INLINE_MATH_41***)**, and **Methodology (***INLINE_MATH_42***)**.

Q20: Why didn't you refit all 8 section weights during Phase 5 tuning?

Fitting 8 independent parameters on only 30 calibration observations (***INLINE_MATH_43***) carries severe risk of sample-specific overfitting. Cross-validation showed that refitting caused extreme parameter volatility. Freezing the robust candidate weights and tuning only the single scalar ***INLINE_MATH_44*** prevented overfitting and allowed the model to generalize successfully to unseen held-out data.

Q21: What are the three scoring modes available in your application?

1. **TF-IDF Baseline (`tfidf`)**: Production baseline linear model over lexical n-grams (Default).
2. **Hybrid Baseline (`hybrid`)**: Convex blend with ***INLINE_MATH_45*** (50% lexical, 50% semantic).
3. **Tuned Hybrid (`tuned_hybrid`)**: Empirically calibrated model with ***INLINE_MATH_46*** (60% lexical, 40% semantic).

Q22: How does the system handle self-matching exclusion?

When an evaluated proposal provides an explicit `project_id` matching an existing reference project (e.g., `SYN-0001`), that project is excluded from candidate matches so it cannot compare against itself. However, when a student submits an unassigned proposal (`project_id = None` or `"NEW_PROPOSAL"`), exact duplicates against the reference corpus are **NOT** excluded, ensuring near-duplicate alerts fire accurately.

---

4. Explainable AI & Scoring Mechanics

Q23: How is the predicted similarity reconstructed additively in your XAI report?

Because the model intercept is zero (***INLINE_MATH_47***), the predicted similarity is simply the sum of each section's weight multiplied by its similarity score:

BLOCK_MATH_3

The UI displays these signed contributions as a direct breakdown of how the total score was accumulated.

Q24: How does the system categorize sections into "High Overlap" vs. "Differentiating"?

- **High-Overlap Sections**: Effective similarity ***INLINE_MATH_48***. These represent areas of potential duplication or shared scope.
- **Differentiating Sections**: Effective similarity ***INLINE_MATH_49***. These represent unique, novel dimensions of the student's proposal.

Q25: How does the keyword evidence extraction prevent displaying generic filler words?

`is_informative_term()` in `src/explainability/explainer.py` filters terms against two dictionaries:

1. Standard English stop-words (`the`, `with`, `for`).
2. Generic academic boilerplate filler (`using`, `proposed`, `system`, `methodology`, `project`, `approach`). It preserves compound technical n-grams (*convolutional neural network*, *plantvillage dataset*).

Q26: How are verbatim context snippets extracted?

`extract_snippet()` locates the exact match of the keyword in the text, expands a 45-character boundary window on either side, snaps to nearest word boundaries so words are not cut in half, and adds ellipses (`...`) only when text was actually truncated.

Q27: What novelty assessment tiers does the system report?

- ***INLINE_MATH_50*** **(Similarity *INLINE_MATH_51*)**: *High / Exceptional Novelty*

- ***INLINE_MATH_52* (Similarity *INLINE_MATH_53*)**: *Moderate Novelty (Partial Overlap)*
- ***INLINE_MATH_54* (Similarity *INLINE_MATH_55*)**: *Moderate-to-Low Novelty (Substantial Overlap)*
- ***INLINE_MATH_56* (Similarity *INLINE_MATH_57*)**: *Low Novelty (High Overlap / Duplicate)*

---

5. Software Architecture & Engineering

Q28: What tech stack was chosen and why?

- **Backend**: **FastAPI** (Python 3.12). Provides asynchronous endpoints, Pydantic validation, automatic Swagger documentation, and high performance.
- **Frontend**: **React 19** with **Vite** and **Tailwind CSS**. Provides responsive SaaS dashboard UI, instant reactive state, and sub-second compilation.
- **Database**: **SQLite 3**. Embedded relational database requiring zero external service dependencies, perfect for verifiable institutional storage.
- **ML / NLP**: **Scikit-Learn**, **PyTorch**, and **Sentence-Transformers**. Industry standard libraries for vectorization and transformer inference.

Q29: What is the purpose of FastAPI's lifespan context manager in your application?

Loading heavy machine learning models (like Sentence-Transformers and 8 TF-IDF vectorizers) takes 2 to 3 seconds. Using FastAPI's `lifespan` handler, models are loaded once into RAM when the server process starts. Subsequent API requests execute in 100 to 350 milliseconds without reloading models.

Q30: Why is scoring decoupled from saving in your API design?

Evaluating a proposal (`POST /api/score`) is strictly **read-only and idempotent**. It scores in-memory without touching the database. If scoring wrote immediately to the database, incomplete drafts, testing experiments, and rejected attempts would pollute institutional submission archives. Persistence (`POST /api/save`) only occurs when explicitly committed by the evaluator.

Q31: How do you prevent duplicate proposals from being inserted into SQLite?

1. **Title Normalization**: Lowercasing, trimming, and punctuation collapsing.
2. **Content Hash**: Generating a deterministic SHA-256 fingerprint over the concatenated 8 normalized sections.
3. If an incoming proposal matches an existing record's normalized title or content hash, the API raises a custom `DuplicateProposalError` which returns an HTTP 409 Conflict with details of the existing project.

Q32: What database tables are created in SQLite?

Two tables:

1. `proposals` : Stores submission metadata, the 8 sections, novelty percentage, max similarity score, and SHA-256 content hash.
2. `evaluation_reports` : Stores the foreign key link to `proposals.id` , scoring mode, effective alpha, and the complete serialized JSON XAI evaluation payload.

Q33: How does the React frontend communicate with the backend?

Through a centralized API service module (`frontend/src/services/api.js`). It handles fetch requests, appends query parameters (`?mode= ... &alpha= ...`), unpacks JSON responses, and standardizes error handling for HTTP 400, 404, 409, and 422 codes.

Q34: How do you handle frontend dirty state when an evaluated proposal is modified?

If a user scores a proposal and subsequently edits any text field in the form, `isDirtyAfterScoring` is set to `true`. This displays a visual warning banner and disables the "Save Proposal" button until the user clicks "Evaluate Novelty" again, preventing mismatched reports from being saved.

---

6. Evaluation Metrics & Scientific Protocol

Q35: What evaluation metrics did you use to compare models?

- **Mean Absolute Error (MAE)**: Average absolute difference between model prediction and human rating: ***INLINE_MATH_58***.
- **Root Mean Squared Error (RMSE)**: Penalizes large outlier errors: ***INLINE_MATH_59***.
- **Mean Bias Error (MBE)**: Direction of error; positive means the model overpredicts similarity, negative means underprediction.
- **Exact Agreement %**: Percentage of test pairs where rounded prediction equals human rating.
- **Within-One Agreement %**: Percentage of test pairs where prediction is within ***INLINE_MATH_60*** rating point of human rating.
- **Macro F1 Score**: Unweighted harmonic mean of Precision and Recall across all 5 discrete rating classes (***INLINE_MATH_61*** to ***INLINE_MATH_62***).

Q36: Why is Macro F1 a more critical metric than simple accuracy in this project?

Academic evaluation datasets are naturally **imbalanced**. Class 3 (High Similarity) had 54 pairs, while Class 2 had only 5 pairs and Class 4 had only 5 pairs. A naive model that always predicts Class 3 could achieve high accuracy while failing completely on true duplicates and original projects. Macro F1 treats all 5 classes equally, penalizing models that fail on minority classes.

Q37: How did your Tuned Hybrid model perform on the held-out test cohort (***INLINE_MATH_63***)?

- **Exact Agreement**: ***INLINE_MATH_64*** (76 out of 90 pairs correct), a ***INLINE_MATH_65*** improvement over baseline hybrid.
- **Macro F1**: ***INLINE_MATH_66***, a ***INLINE_MATH_67*** leap over baseline hybrid (***INLINE_MATH_68***).
- **Within-One Agreement**: ***INLINE_MATH_69*** (89 out of 90 pairs within ***INLINE_MATH_70*** point).
- **Continuous RMSE**: ***INLINE_MATH_71***.
- **Spearman Rank Correlation**: ***INLINE_MATH_72***.

Q38: What is Quadratic Weighted Kappa (QWK) and why did you report it?

QWK (***INLINE_MATH_73***) measures inter-rater agreement for **ordinal data**, penalizing errors quadratically based on distance:

BLOCK_MATH_4

An error between Rating 0 and Rating 1 receives a penalty of ***INLINE_MATH_74***, whereas an error between Rating 0 and Rating 4 receives a penalty of ***INLINE_MATH_75***. Our Tuned Hybrid model achieved a QWK of ***INLINE_MATH_76***, indicating near-perfect agreement.

Q39: What is the difference between Model-vs-Reviewer Agreement and Inter-Reviewer Reliability?

- **Model-vs-Reviewer Agreement**: Measures how closely our algorithm matches the ratings given by one human annotator (`Reviewer 1`).

- **Inter-Reviewer Reliability (IRR)**: Measures how consistently **two different human experts** agree with each other on the same proposals (e.g., Cohen's Kappa between Reviewer 1 and Reviewer 2). Because Reviewer 2 is currently pending, IRR cannot yet be calculated.

Q40: How many automated tests exist in your test suite and what do they cover?

114 automated tests (all passing, 100% pass rate):

- Integration tests (14 tests) covering 7 controlled scenarios.
- Hybrid scoring and formula tests (16 tests).
- Phase 5 preliminary evaluation validation (9 tests).
- Annotation workbook validation (17 tests).
- API backend endpoint tests (10 tests).
- Candidate constrained model boundary tests (9 tests).
- Scientific reliability tests (10 tests).
- SQLite storage and deduplication tests (14 tests).
- XAI attribution and snippet tests (15 tests).

---

7. Challenging External Examiner Questions

Q41: "Why did you use Ridge Regression and SLSQP instead of fine-tuning an end-to-end Deep Transformer like RoBERTa or BERT?"

> **Answer**: "End-to-end transformer fine-tuning requires thousands of labeled proposal pairs to avoid catastrophic forgetting and overfitting. We only had 120 faculty-annotated pairs. Training a massive neural network would lead to severe overfitting. Furthermore, an end-to-end transformer is a black box that cannot produce verified section-level additive attributions. Our modular architecture uses frozen pre-trained embeddings for semantic representation and a constrained linear head for fusion. This achieves high sample efficiency, zero overfitting risk, and complete mathematical explainability."

Q42: "If your model achieved 84.44% exact agreement, why didn't you make Tuned Hybrid the default mode in the API?"

> **Answer**: "This was a deliberate academic governance decision. The tuning was performed against ratings from a **single faculty reviewer** (`Reviewer 1`). Until independent double-review by Reviewer 2 and panel adjudication are finalized, promoting a tuned model to production default would violate scientific integrity protocols. We preserve the production-proven TF-IDF model as the default, while providing Tuned Hybrid as a transparently documented, selectable option."

Q43: "What happens if a student writes a proposal with completely scrambled sections—putting the dataset in the abstract and methodology in the problem statement?"

> **Answer**: "Because our system relies on structured sections, placing text in the wrong section degrades section-level cosine alignment. However, university project proposals are submitted through formal departmental portals with distinct mandatory input fields, ensuring section boundaries are maintained. Furthermore, the semantic branch still captures topical vocabulary across the proposal."

Q44: "How does your system scale if the historical corpus grows from 600 projects to 100,000 projects?"

> **Answer**: "Currently, scoring compares the candidate against all 600 projects in ~300ms using matrix vectorization in NumPy. For 100,000 projects:
>
> 1. We would introduce an approximate nearest neighbor (ANN) vector indexing library like **FAISS** or **HNSW** for the dense embeddings.
> 2. We would implement a coarse two-tier filtering strategy: retrieve the top 200 candidate matches via fast sparse BM25/TF-IDF indexing, and then run full multi-section hybrid scoring and XAI explanation strictly on those top 200 candidates. This scales logarithmically: ***INLINE_MATH_77***."

Q45: "Why did you assign zero weights to Abstract and Problem Statement? Isn't the problem statement important in research?"

> **Answer**: "In conceptual research, the problem statement is critical. However, in our quantitative feature correlation analysis, student problem statements often use generic societal justifications—such as *'heart disease is the leading cause of mortality worldwide*'. Multiple completely unrelated projects shared high cosine similarity in problem statements due to shared public health statistics. In SLSQP optimization, assigning weights to problem statements produced false-positive similarity spikes across unrelated projects. The real differentiating dimensions that prove duplication in computer science capstones are the **Dataset**, the **Title**, and the **Methodology**."

Q46: "Could a student game your system by replacing the dataset name with a fake synthetic name while keeping everything else identical?"

> **Answer**: "If a student replaces the dataset name, the dataset similarity drops to ***INLINE_MATH_78***. However, the candidate constrained model assigns ***INLINE_MATH_79*** (***INLINE_MATH_80***) to Title, ***INLINE_MATH_81*** (***INLINE_MATH_82***) to Technology, and ***INLINE_MATH_83*** (***INLINE_MATH_84***) to Methodology. The combined remaining sections will still accumulate a similarity score of ***INLINE_MATH_85*** out of ***INLINE_MATH_86***. This places the proposal in the *Moderate-to-Low Novelty* tier, alerting the faculty panel to investigate methodological duplication even if the dataset was obscured."

Q47: "What is data leakage and how did your experimental setup strictly prevent it?"

> **Answer**: "Data leakage occurs when information from the test/held-out evaluation cohort contaminates the training or hyperparameter tuning process, producing artificially optimistic results. We strictly prevented leakage by:
>
> 1. Partitioning the 120 faculty pairs into 30 DEV pairs and 90 HELD-OUT pairs before conducting any tuning.
> 2. Alpha grid search and section weight analyses were executed strictly using 5-fold cross-validation on the 30 DEV pairs.
> 3. The configuration (***INLINE_MATH_87***) was frozen and saved to disk.
> 4. The held-out 90 pairs were unblinded and evaluated **exactly once**. Zero parameters or weights were adjusted after seeing the held-out metrics."

Q48: "What is Mean Bias Error (MBE) and why was it an important metric during your alpha tuning?"

> **Answer**: "MAE and RMSE measure the magnitude of errors, but they square or take the absolute value, losing the error direction. Mean Bias Error (***INLINE_MATH_88***) indicates whether the model has a systematic tendency to overpredict (***INLINE_MATH_89***) or underpredict (***INLINE_MATH_90***). At ***INLINE_MATH_91***, the model had a positive bias of ***INLINE_MATH_92***, constantly overpredicting similarity because dense embeddings find background resemblance. At ***INLINE_MATH_93***, the model had a negative bias of ***INLINE_MATH_94***, underpredicting similarity. ***INLINE_MATH_95*** achieved an almost perfectly balanced bias of ***INLINE_MATH_96***."

Q49: "How do you justify using a single reviewer's ratings as ground truth?"

> **Answer**: "We do not represent single-reviewer ratings as definitive ground truth. In our documentation and research reports, we explicitly classify our results as **preliminary human evaluation**. Due to the faculty member's co-reviewer being temporarily unavailable, we utilized the completed 120 ratings from Reviewer 1 to validate that our tuning methodology was sound. Independent double-review by Reviewer 2 and formal panel adjudication are established as the immediate next phase before institutional deployment."

Q50: "What is your main takeaway from this project as a Computer Science engineer?"

> **Answer**: "My main takeaway is that in real-world applied AI, **explainability, mathematical boundedness, and rigorous evaluation methodology are far more important than raw model complexity**. Building a complex black-box deep network that outputs a single similarity number would have failed in an academic setting where faculty need to justify decisions. By designing a constrained, section-weighted hybrid architecture with transparent additive attribution and zero leakage, we built an engineering solution that is scientifically sound, computationally efficient, and directly usable by academic institutions."
